COS341 Semester Project (P2): User Manual

Group 13

Name	Student number
Ayush Beekum	u23596351
Jaitin Moodally	u23621372
Kgaugelo Matsena	u23658462
Lusanda Mtembu	u23602016

Submission category: 1st Class (Parts 2a, 2b, 3a and 3b, including the generation of executable target code). See the signed declarations in Section 8.

1. Requirements

- Java 17 or higher, to run the compiler.
- **g++** (preferred, tested with g++ 13) or gcc 13, to compile the generated target code.

2. Launching the compiler

From the directory containing the jar:

```
java -jar COCO-Group13.jar                  # reads SPL.txt from the current directory
java -jar COCO-Group13.jar <path to file>   # reads the given SPL source file
```

If no argument is given and there is no `SPL.txt` in the current directory, the compiler reports that it cannot read the input file.

3. What the compiler does and which files it writes

All output files are written into the directory the jar is run from.

Phase	On success	On error
Lexer and parser (P1)	tree.xml (syntax tree)	`Lexical error` or `Syntax error` on stderr, nothing written
Scope analysis (2a) Type analysis (2b)	symbol_table.txt (internal names v1, f1, ... and types)	`Semantic error` or `Type error` on stderr; no symbol_table.txt, no output.cpp
Code generation (3a, 3b)	**output.cpp** (executable C++ target code)	-

On any error the compiler exits with code 2 and never leaves a stale output file behind.

4. Compiling and running the generated code

The target code uses `std::cout`, so it is C++. **We prefer g++ for the run-tests:**

```
g++ output.cpp -o program
./program
```

gcc 13 also works, provided the C++ standard library is linked:

```
gcc output.cpp -lstdc++ -o program
./program
```

The generated code was tested with g++ 13 under `-std=c++11`, `-std=c++17`, `-std=c++20` and `-pedantic-errors`, and with gcc 13 as shown above. Every `print` ends its screen line (`<< std::endl`), so that consecutive outputs do not run together.

5. Structure of the generated code

The code-mine (non-executable) comes first, between `/* begin of the non-executable code-mine for later inlining */` and `/* end of the non-executable code-mine */`. It lists every function with its parameters, its body and, for num functions, its return variable r. Every line of the mine is a `//` C comment, because the function bodies themselves contain `/* ... */` comments and C block comments cannot be nested.

The executable part follows: `#include <iostream>`, `int main()`, the declarations of the main program's variables, the translated algorithm with all function calls inlined, and `return 0 ;`. As required, a new line follows every semicolon.

Name in target code	Meaning
v1, v2, ...	SPL variables and parameters (internal names from the symbol table of phase 2a)
t1, t2, ...	temporary variables created by the code generator (as in textbook Fig.6.3)
L1, L2, ...	jump labels
f1t7	return value r = f ++ n of function f1
X_k	name X belonging to the k-th inlined copy of a function

The number type is chosen once for the whole program, following the rules of phase 2b: `int` if the program uses mod, `float` if it uses div or contains a decimal number, and `int` otherwise.

6. Design decisions

- **Boolean conditions.** trans(BOOL, T, F) always jumps either to T (true) or to F (false) and never falls through: every comparison becomes `if ( x op y ) goto T ; goto F`. This is textbook Fig.6.8 with each ELSE replaced by a goto. *not* is then exactly the label swap of the specification; *or* and *and* use one intermediate label. We deviate from the literal rules of sheet 3a because they generate wrong code: when the false case simply falls through to the else-label, `if not ( eq ( #a #b ) ) ...` enters the else-branch in both cases, and the given *and* rule expands to the same code as *or*. The target syntax is unchanged: only `if ( x op y ) goto L` with op in == > <, `goto L` and labels; no else, and, or, not.
- **Loops** follow the sheet: until-loop and do-while-loop directly, while and do-until by the program transformations with not (which here is just the label swap).
- **Parameter passing.** For a call, each argument is evaluated into a fresh variable p1 ... pN and then assigned to the formal parameter (`a1 = p1`), which is the intended meaning of the line "p1 = a1" on sheet 3b.
- **Inlining with renaming.** Each inlined copy renames all variables, temporaries and labels the function declares (suffix _k for the k-th copy). This includes copies of its sub-functions, which were already inlined into it inside the code-mine. So repeated calls, such as Collatz called twice, never produce duplicate labels or declarations. Names of enclosing scopes and globals are not renamed.
- **Calls with a different number of arguments** are accepted by the phase 2 rules. Arguments and parameters are paired in order; extra arguments are evaluated and ignored, and missing ones leave the parameter uninitialised.
- **Declarations** of temporaries have no initialiser, so a goto may legally jump over them in C++.

7. Example

SPL source (`SPL.txt`):

```
#x :
num #dbl ( #a ) { : : nop ; return ( add ( #a #a ) ) }
:
#x = #dbl ( 3 ) ;
if larger ( #x 5 ) then { print "big" ; } else { print ( #x ) ; } ;
```

Generated `output.cpp` (it prints `big`):

```
/* begin of the non-executable code-mine for later inlining */
// begin of code-mine for Function f1
// Parameters for inlining:
// int v2 ;
// Body of the procedure:
// /* nop */ ;
// /* end */ ;
// int f1t1 ;
// int t2 ;
// int t3 ;
// t2 = v2 ;
// t3 = v2 ;
// f1t1 = t2 + t3
// end of code-mine for Function f1:
// The result is found in the return value f1t1:
/* end of the non-executable code-mine */
/* begin of executable C code */
#include <iostream>
int main()
{
    int v1 ;
    int t4 ;
    /* begin inlining f1 */ ;
    int t5 ;
    t5 = 3 ;
    int v2_1 ;
    v2_1 = t5 ;
    /* nop */ ;
    /* end */ ;
    int f1t1_1 ;
    int t2_1 ;
    int t3_1 ;
    t2_1 = v2_1 ;
    t3_1 = v2_1 ;
    f1t1_1 = t2_1 + t3_1 ;
    t4 = f1t1_1 ;
    /* end inlining f1 */ ;
    v1 = t4 ;
    /* if */
    int t6 ;
    t6 = v1 ;
    int t7 ;
    t7 = 5 ;
    if ( t6 > t7 ) goto L1 ;
    goto L2 ;
    L2 : /* else */
    int t8 ;
    t8 = v1 ;
    std::cout << t8 << std::endl ;
    /* end */ ;
    goto L3 ;
    L1 : /* then */
    std::cout << "big" << std::endl ;
    /* end */ ;
    L3 : /* continue */ ;
    /* end */
    return 0 ;
} /* end of executable C */
```

8. Declarations

Declaration 1: Submission category

We, the undersigned members of Group 13, declare that we submit this project in the **"1st Class" category** (maximally 6/6 points), comprising Semantic Analysis (Part 2a), Type Checking (Part 2b) and the generation of executable target code (Parts 3a and 3b). All members of the group agree with this choice of category.

Name	Student number	Signature	Date
Ayush Beekum	u23596351		
Jaitin Moodally	u23621372		
Kgaugelo Matsena	u23658462		
Lusanda Mtembu	u23602016		

Declaration 2: Contents of the ZIP file

We, the undersigned members of Group 13, declare that every member of the group is fully aware of the whole contents of the submitted ZIP file, and agrees with the submission of the ZIP file in this form.

Name	Student number	Signature	Date
Ayush Beekum	u23596351		
Jaitin Moodally	u23621372		
Kgaugelo Matsena	u23658462		
Lusanda Mtembu	u23602016		